Pivot Integer

Finding the Pivot Integer using Pointers in C

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads a positive integer n. The value of n must be passed to a function named pivot_integer() using its address, and pointers must be used to pass the value between the calling function and pivot_integer(). The function must find the pivot integer x between 1 and n such that the sum of all integers from 1 to x (inclusive) is equal to the sum of all integers from x to n (inclusive).

- If such an integer x exists, display it.
- If no such integer exists, display **-1**.

2. Examples

- **n = 8:** Pivot integer is 6, because 1 + 2 + 3 + 4 + 5 + 6 = 6 + 7 + 8 (both sides equal 21).
- **n = 1:** Pivot integer is 1, because 1 = 1.
- **n = 4:** Output is -1, because no pivot integer exists.

3. Concepts Used

- **Pointers:** the address of n is passed using the & operator, and the function accesses the value using the dereference operator (*).
- **Functions:** the logic is separated into a user-defined function, pivot_integer().
- **Loops and conditions:** a for loop checks every value of x from 1 to n, and an if statement compares the two sums.
- **Input validation:** the program rejects inputs that are not positive integers.

4. Algorithm

1. Start the program.
2. Read a positive integer n from the user. If the input is invalid, display an error message and stop.
3. Call pivot_integer(&n), passing the address of n.
4. Inside the function, compute total = n(n + 1) / 2, the sum of all integers from 1 to n, using the pointer.
5. Set left_sum = 0. For each x from 1 to n: add x to left_sum (this gives 1 + 2 + … + x), and compute right_sum = total − left_sum + x (this gives x + (x+1) + … + n).
6. If left_sum equals right_sum, display "Pivot integer: x" and return.
7. If the loop ends without a match, display -1.
8. Stop.

5. Mathematical Insight

Let S = n(n + 1) / 2 be the sum of 1 to n. The condition (1 + … + x) = (x + … + n) can be written as x(x + 1)/2 = S − x(x − 1)/2 + x, which simplifies to:

$$x^2 = n(n + 1) / 2$$

So a pivot integer exists only when $n(n + 1)/2$ is a perfect square. For $n = 8$, $8 \times 9 / 2 = 36 = 6^2$, so $x = 6$. For $n = 4$, $4 \times 5 / 2 = 10$ is not a perfect square, so the output is -1. The program uses a loop that follows the definition directly.

6. Source Code

File name: pivot_integer.c

```c
#include <stdio.h>

/* Function prototype */
void pivot_integer(int *ptr);

int main(void)
{
    int n;

    printf("Enter a positive integer n: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Invalid input. Please enter a positive integer.\n");
        return 1;
    }

    /* Pass the address of n to the function */
    pivot_integer(&n);

    return 0;
}

/* Finds and displays the pivot integer for *ptr */
void pivot_integer(int *ptr)
{
    long long total, left_sum = 0, right_sum;
    int x;

    /* Sum of all integers from 1 to n */
    total = (long long)(*ptr) * (*ptr + 1) / 2;

    for (x = 1; x <= *ptr; x++)
    {
        left_sum += x;                    /* 1 + 2 + ... + x */
        right_sum = total - left_sum + x; /* x + (x+1) + ... + n */

        if (left_sum == right_sum)
        {
            printf("Pivot integer: %d\n", x);
            return;
        }
    }

    /* No pivot integer found */
    printf("-1\n");
}
```

7. Code Explanation

- **void pivot_integer(int *ptr);** is the function prototype. The parameter is a pointer to int.
- **scanf("%d", &n)** reads the value; the condition n <= 0 rejects non-positive input.
- **pivot_integer(&n)** passes the address of n, so ptr points to the same memory location as n.
- ***ptr** gives the value of n inside the function.
- **long long** is used for the sums so that large values of n do not overflow.
- **return;** inside the loop exits the function as soon as the pivot integer is found, so -1 is not printed.

8. Sample Output

Test Case 1: n = 8

```
Enter a positive integer n: 8
Pivot integer: 6
```

Test Case 2: n = 1

```
Enter a positive integer n: 1
Pivot integer: 1
```

Test Case 3: n = 4

```
Enter a positive integer n: 4
-1
```

Test Case 4: invalid input (n = 0)

```
Enter a positive integer n: 0
Invalid input. Please enter a positive integer.
```

9. Test Case Summary

Input (n)	Output	Explanation
8	Pivot integer: 6	1+2+3+4+5+6 = 21 = 6+7+8
1	Pivot integer: 1	1 = 1
4	-1	No pivot integer exists for n = 4
49	Pivot integer: 35	$x^2 = 49 \times 50/2 = 1225$, so x = 35
10	-1	$10 \times 11/2 = 55$ is not a perfect square

10. Conclusion

The program correctly finds the pivot integer for a given positive integer n, or prints -1 when none exists. It demonstrates the use of pointers for passing a variable's address to a function, dereferencing a pointer to read its value, loops, conditions and input validation. All test cases, including the three given examples, produced the expected output.